Lecture 13 – CARTs and Ensemble Methods

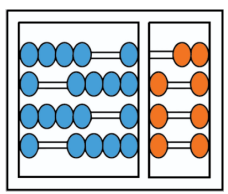
[ISLP 8.1–8.2]

MC886 – Machine Learning

Marcelo S. Reis †

† Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estatual de Campinas

Campinas, April 13, 2026



Instituto de Computação

UNIVERSIDADE ESTADUAL DE CAMPINAS

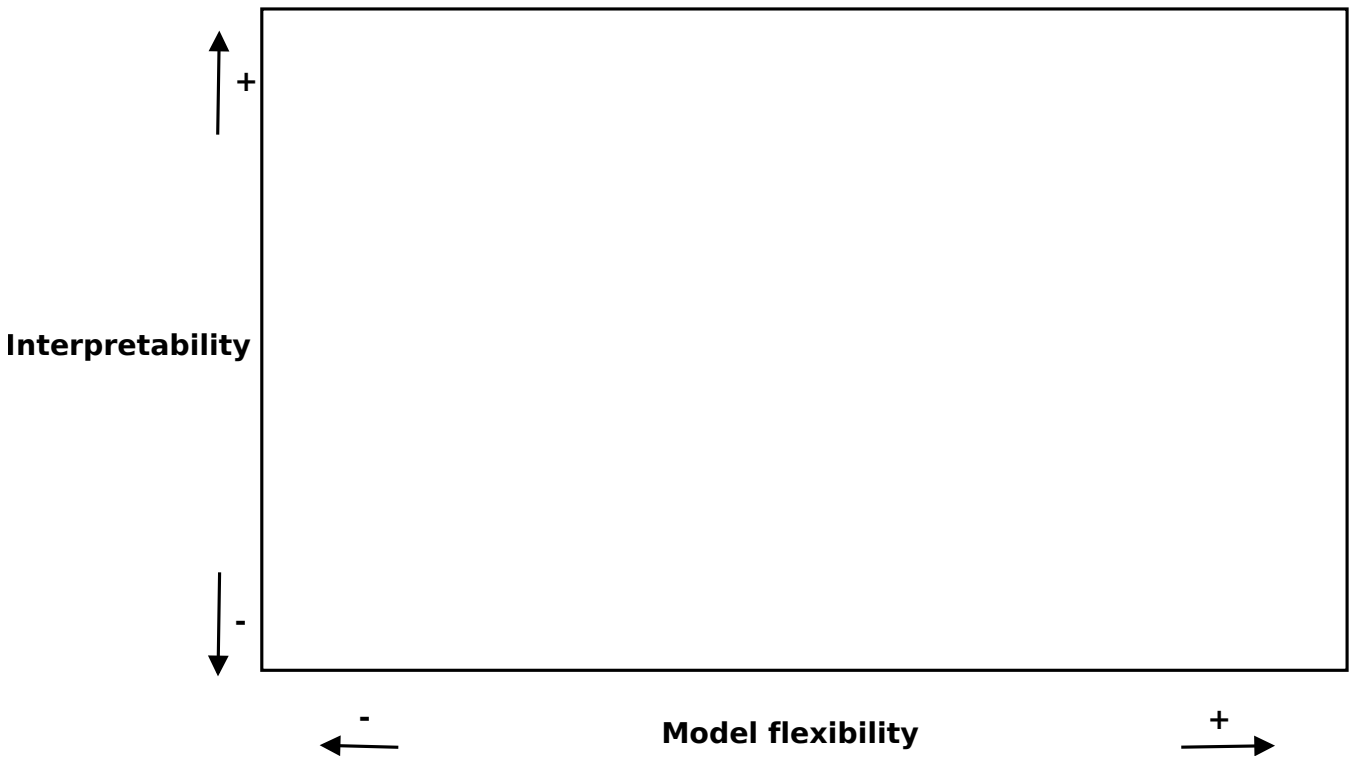
Today's summary

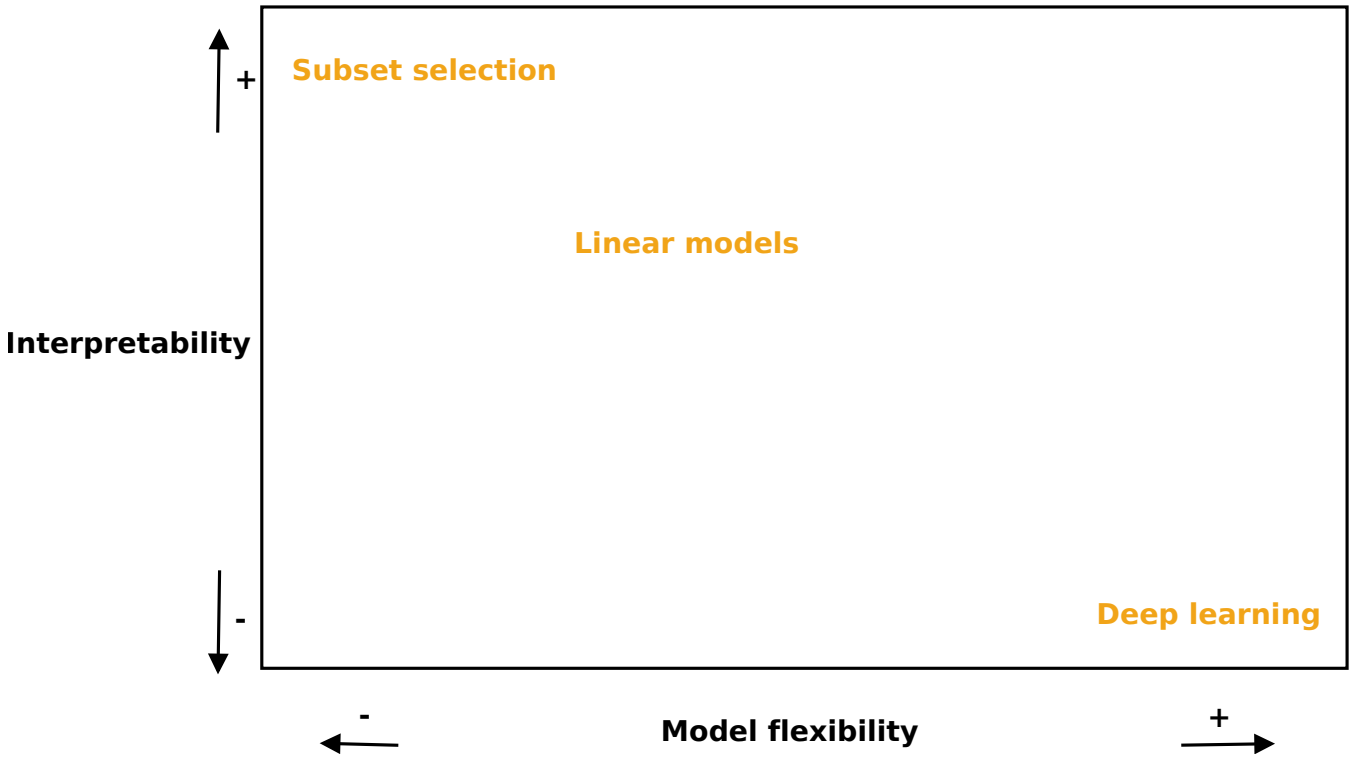
Classification and regression trees (CARTs)

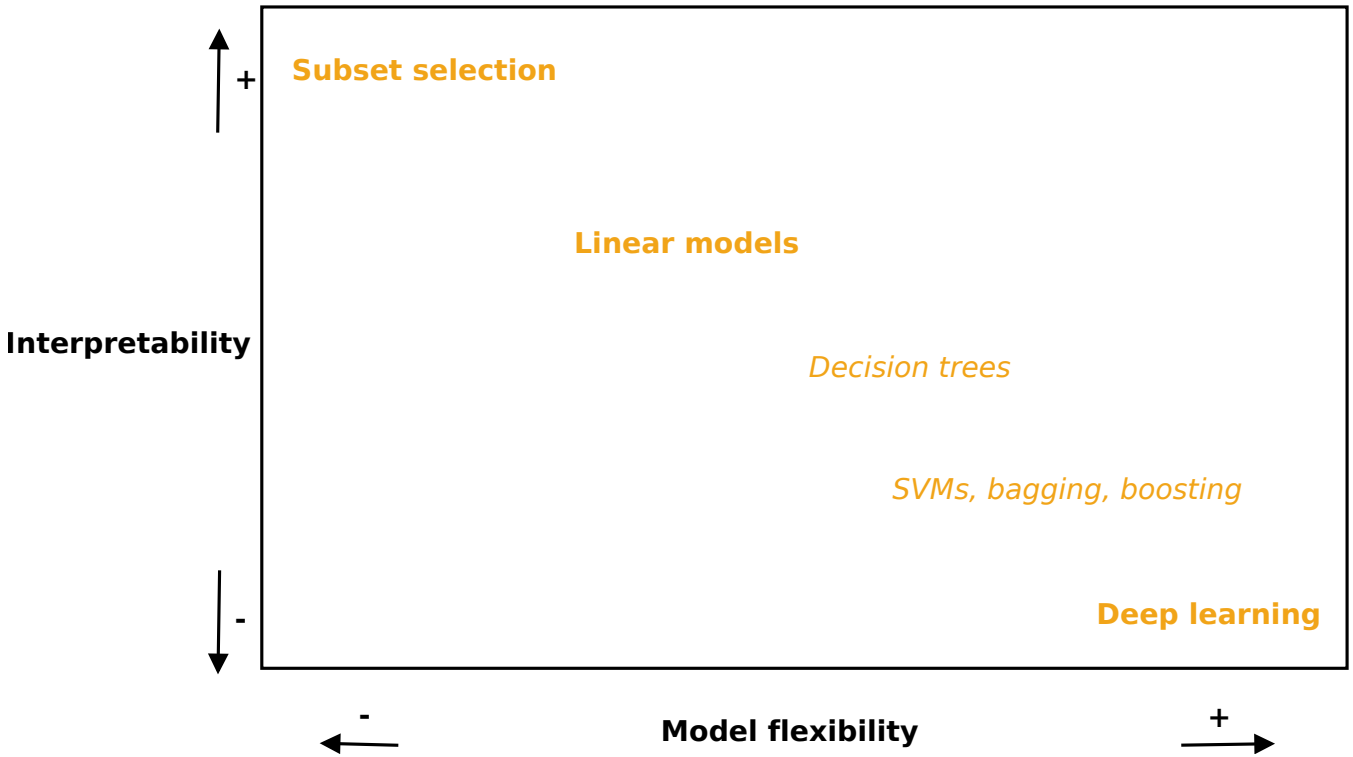
Learning of CARTs

Ensemble learning: "Unity makes strengh!"

Bagging, Random Forest, Boosting


+
Interpretability
-
-
Model flexibility
+


+
Subset selection
Linear models
Interpretability
-
Deep learning
-
Model flexibility
+


+
Subset selection
Linear models
Interpretability
Decision trees
SVMs, bagging, boosting
-
Deep learning
-
Model flexibility
+

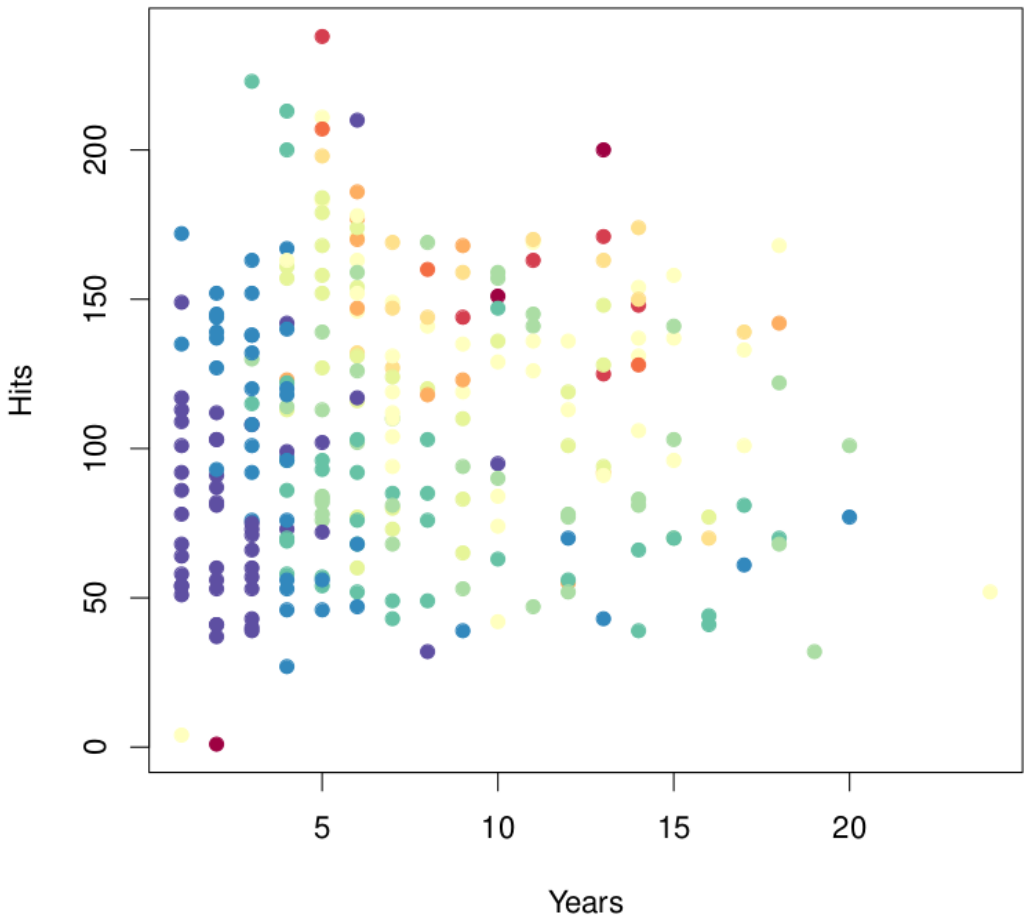
Classification and regression trees (CARTs)

- They work based on *segmentation* of the input space $\mathcal{X}$ into a number of simple regions
- It is done through a set of splitting rules
- The *decision tree* summarizes all possible sequences of sppliting rules for a given segmentation

More on decision trees

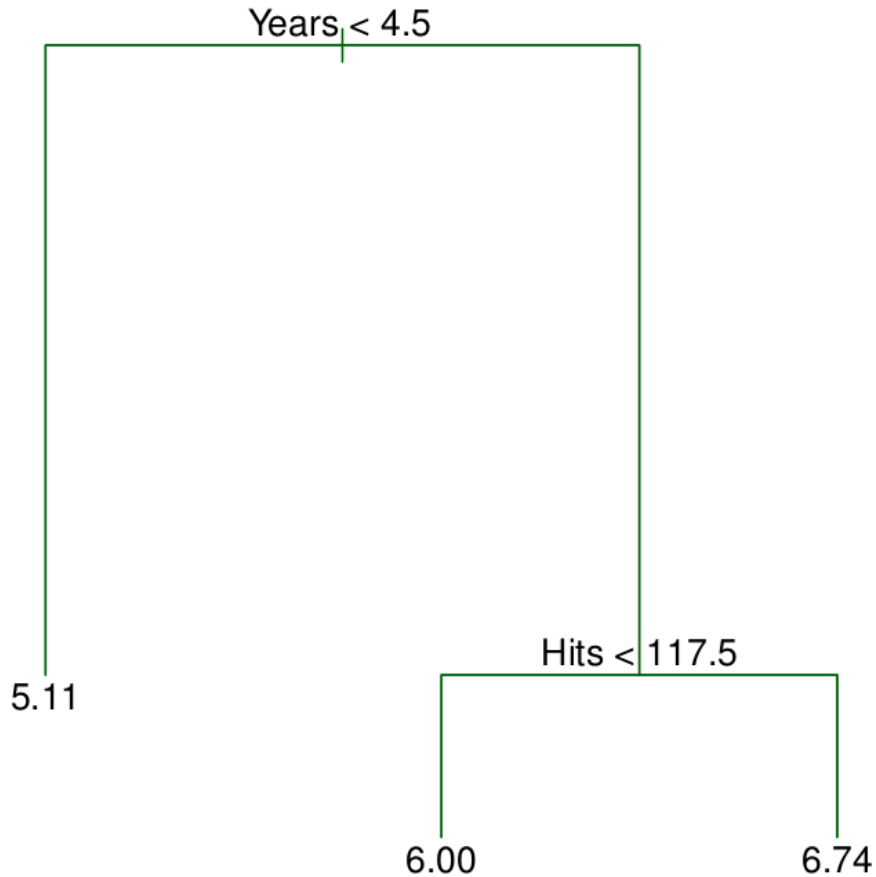
- They are simple and of easy interpretation
- They can be used for both regression and classification problems
- Let's first see them working in regression

Example: Baseball dataset



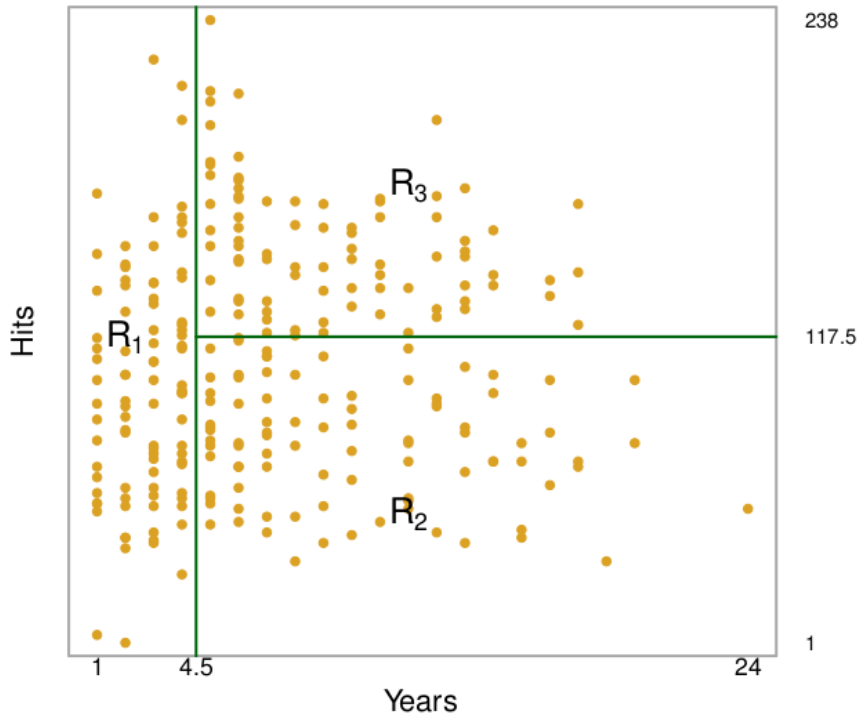
Salary goes from low (blue, green) to high (yellow, red)

Decision tree for `Baseball` dataset



- Leaf values are the mean of the response for observations that fall there

Yielded segmentation for Baseball dataset

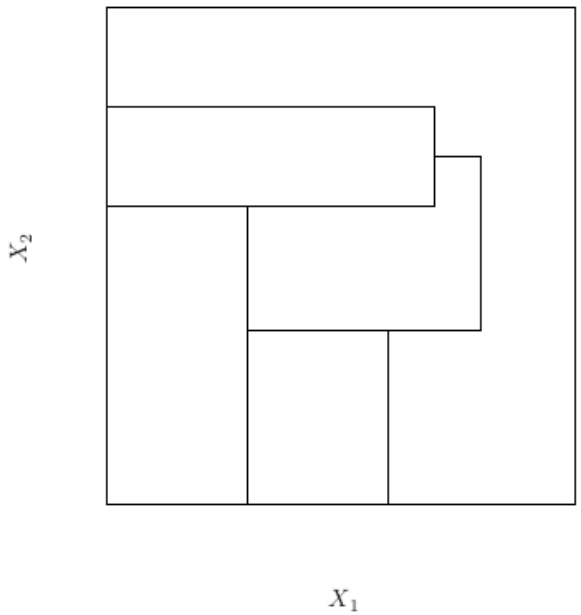


- The tree segments the players into three regions of $\mathcal{X}$:
  - $R_1 = \{x | \text{Years} < 4.5\}$
  - $R_2 = \{x | \text{Years} \geq 4.5 \text{ and Hits} < 117.5\}$
  - $R_3 = \{x | \text{Years} \geq 4.5 \text{ and Hits} \geq 117.5\}$

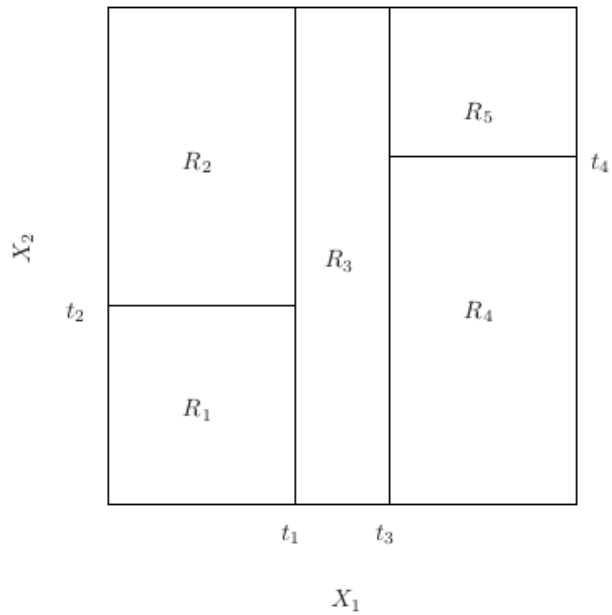
Basics of decision trees

- We split $\mathcal{X}$ into J non-overlapping regions $R_1, R_2, \ldots, R_J$
- For every observation (data point) that falls in R_j we make the same prediction (the mean of response values for the training observations in R_j)
- We generally segment $\mathcal{X}$ into a set of R_j high-dimensional rectangles (boxes), which make model intepretation easier

Data segmentation into boxes



Non-box segmentation



Box segmentation

Learning of CARTs

The goal is to find boxes $R_1, R_2, \ldots, R_J$ such that:

$$\min_{R_1,\ldots,R_J} \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2, \tag{8.1}$$

where $\hat{y}_{R_j}$ is the mean response for all data points in the jth box R_j

- Procedure for learning the optimum set of boxes?

Learning $R_1, R_2, \ldots, R_J$

- Consider all possible partitions is not computationally feasible
- We need to use some suboptimal procedure
- A common one is a top-down, greedy approach

Top-down, greedy approach for learning the regression tree

1. Select variable x_j and cutpoint s_1 such that splitting $\mathcal{X}$ into $\{x|x_j < s_1\}$ and $\{x|x_j \geq s_1\}$ leads to the largest possible reduction in Eq. 8.1

2. In second iteration, we look for the x_k and cutpoint s_2 that splits **one of the previous two regions** in a way that leads to the greatest possible reduction in Eq. 8.1

3. This process continues until a stop criterion is achieved (e.g., no R_j has more than five data points)

Testing the trained tree

- The top-down, greedy approach has a shortcoming
  - It might overfit the data (segmentation memorizes the points)
- **One possible remedy:** work with few boxes $R_1, \ldots, R_j$
- This remedy, however, can lead to bad results if we are unluck to have a "bad" split in earlier iterations and latter ones to have "good" ones
- **Better alternative:** regularization through pruning

Regularization through pruning

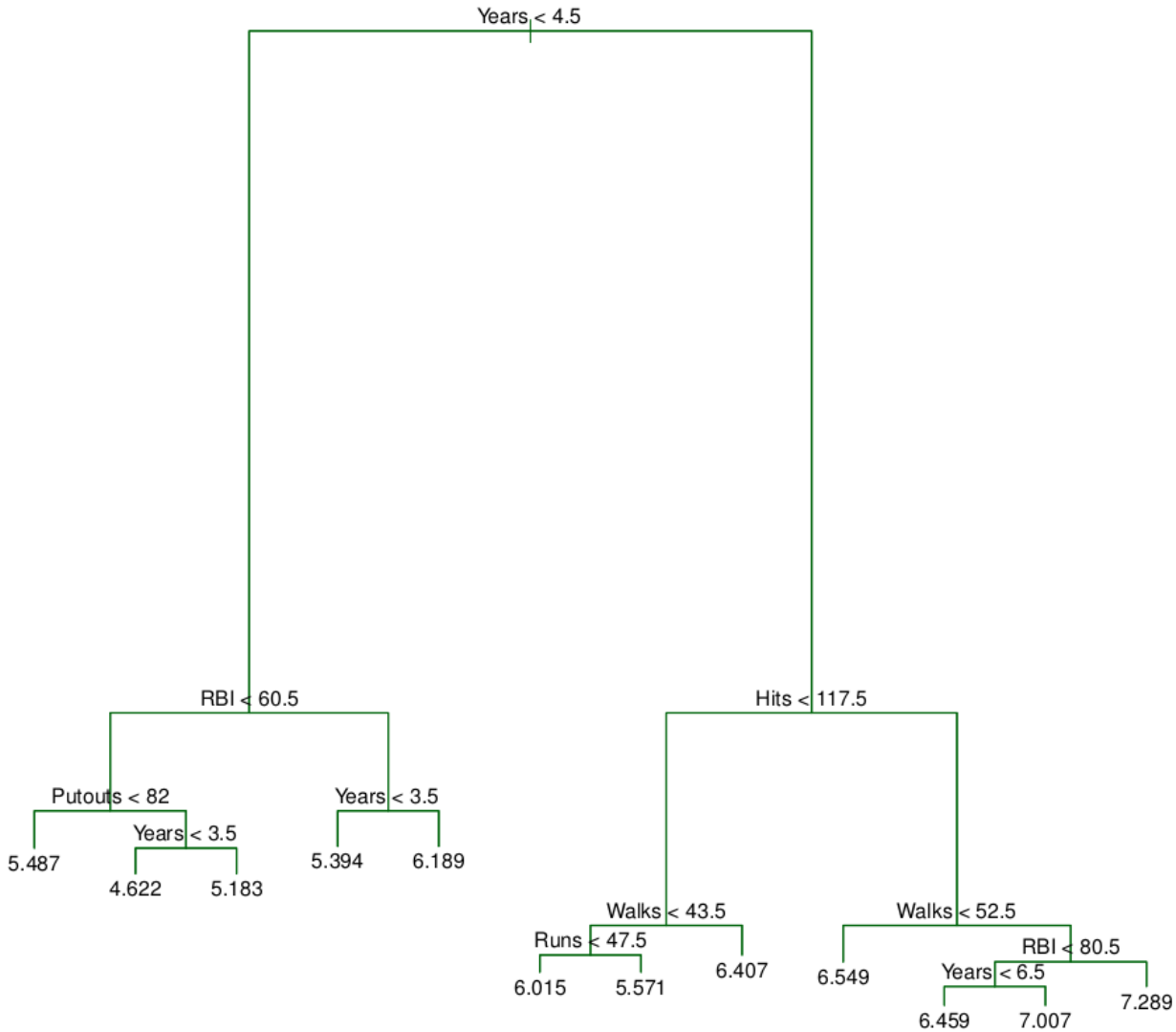
- Let $|T|$ be the number of leaves of tree T
- We compute a tree T_0, where $|T_0|$ is very large, and then prune it back to get a subtree $T \subset T_0$
- Given T_0, the size of T is chosen by minimizing the expression:

$$\min_m \sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|, \tag{8.4}$$

where α is a hyperparameter

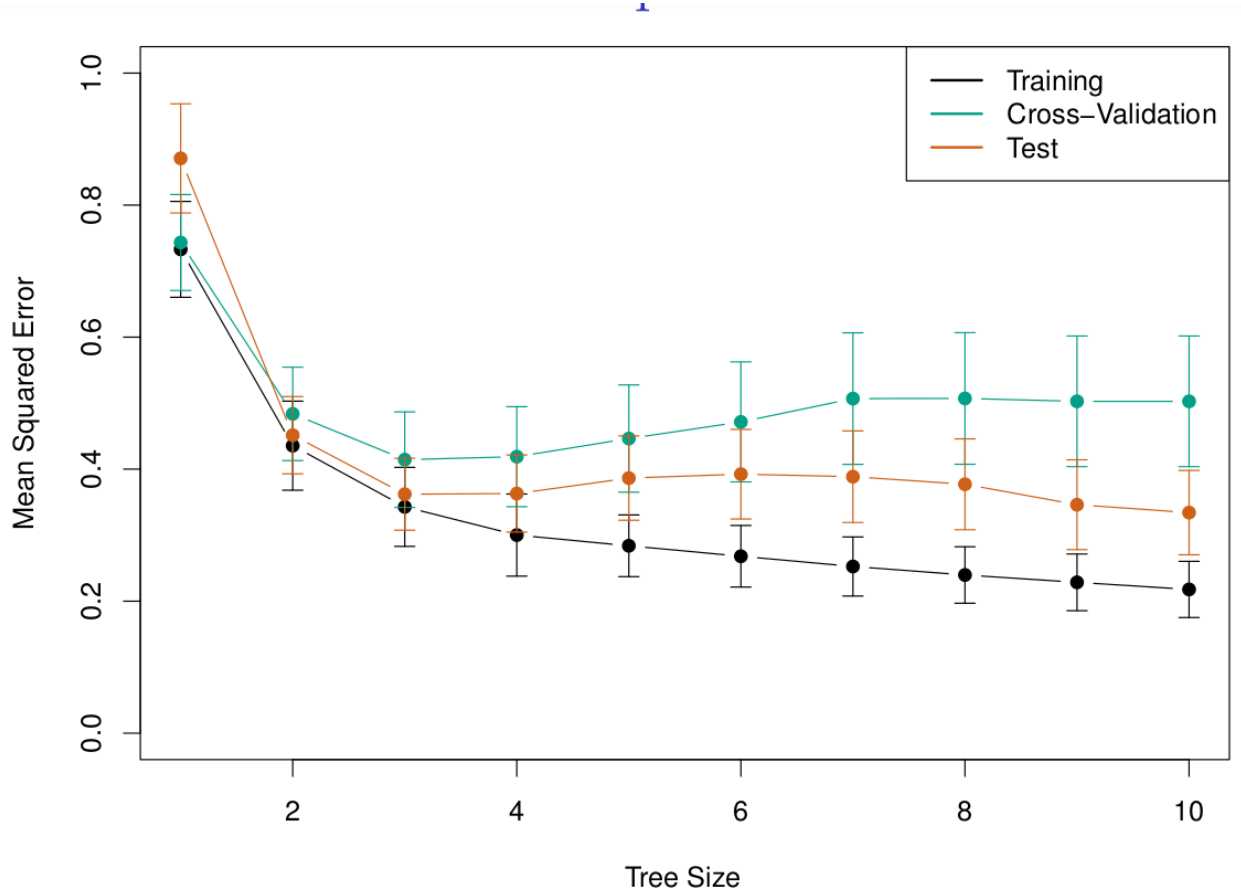
- We can use K-fold CV to choose a suitable value for α

Example: Baseball dataset using more attributes

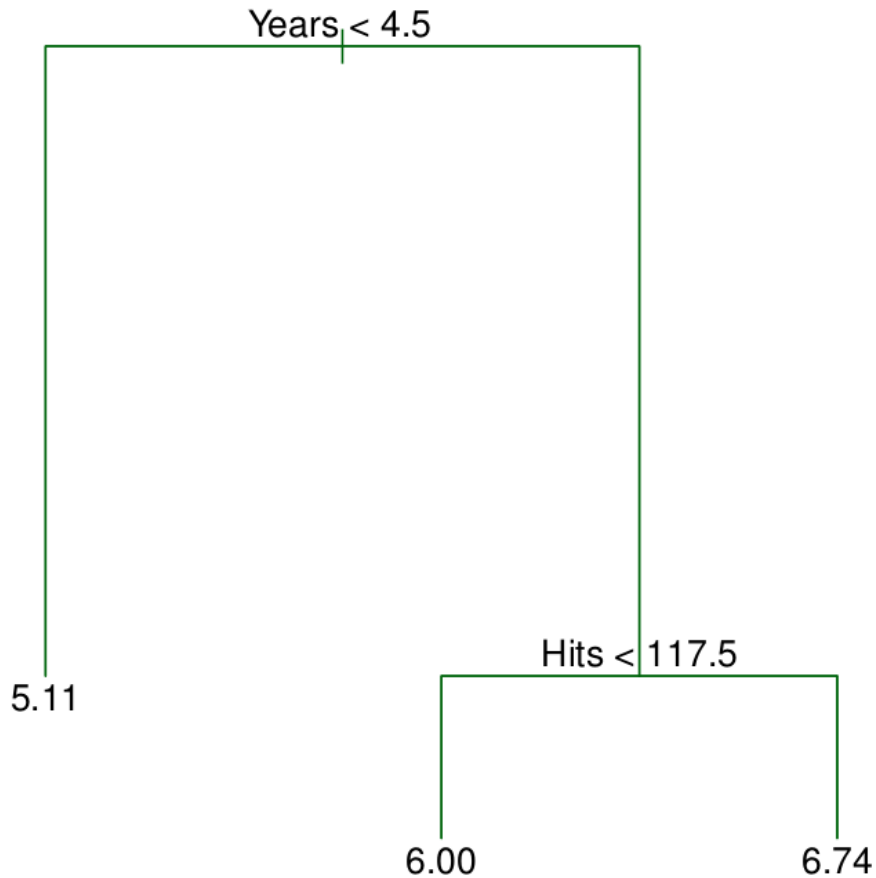


$$x = (\texttt{Hits RBI Runs Putouts Walks Years})^T$$

6-fold CV on Baseball



Chosen pruned tree T ($|T| = 3$)



Classification trees

- Used to predict a qualitative response instead a quantitative one

- We predict that a data point belongs to the most commonly occuring class in the segment R_j to which it belongs

- Instead of Eq. 8.1, we use a classification error rate, defined as the fraction of the training observations that don't belong to the most common class:

$$E = 1 - \max_k(\hat{p}_{mk}), \tag{8.5}$$

where $\hat{p}_{mk}$ is the proportion of data points in mth region that are from the kth class

Two other error measures

- Classification error rate is not sensitive enough for the tree-growing step, so in practive we use other measures

- The *Gini index* is given by:

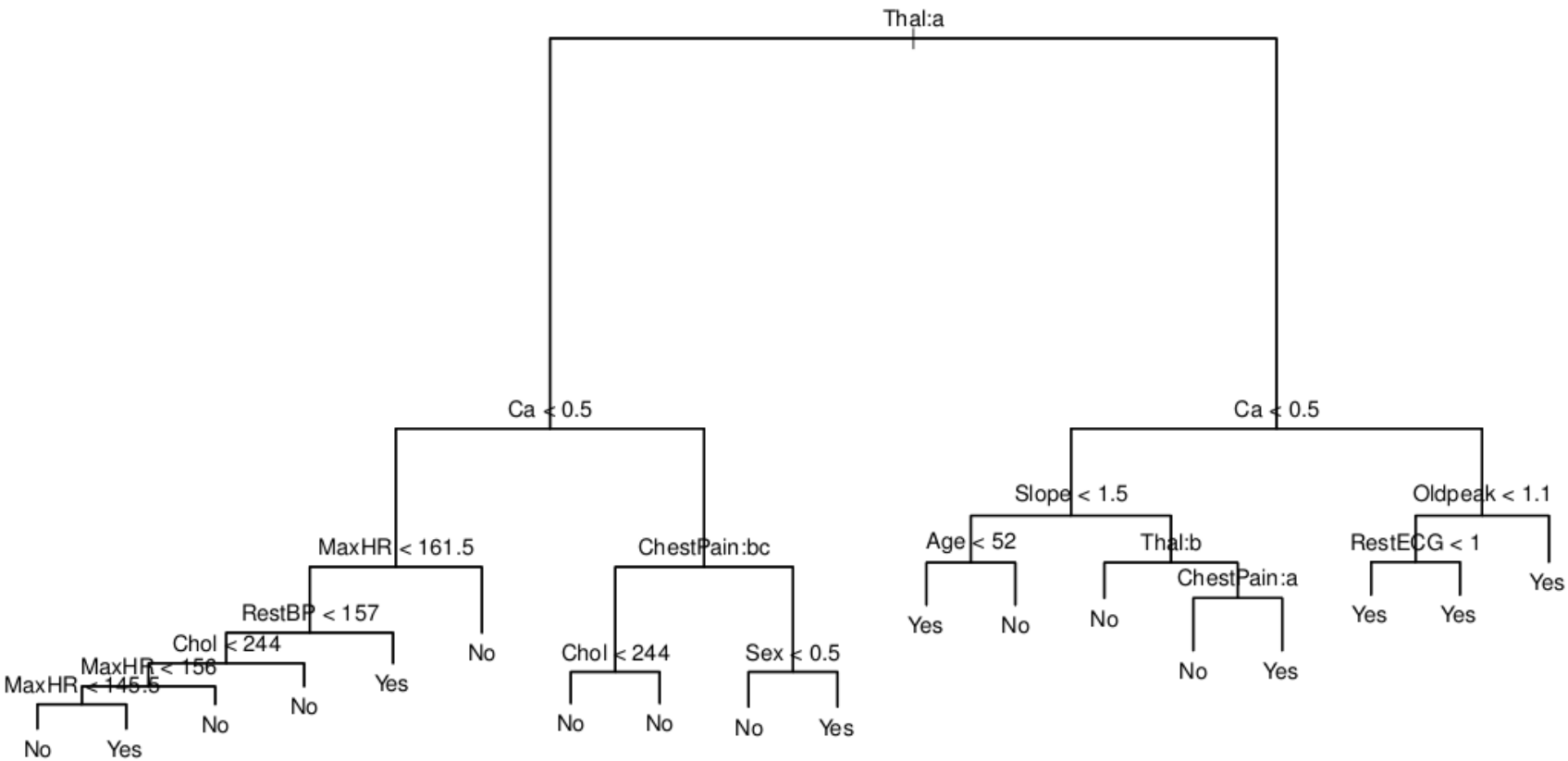
$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}), \tag{8.6}$$

  which is a measure of total variance across the K classes.

- Other possibility is the usage of *cross entropy*:

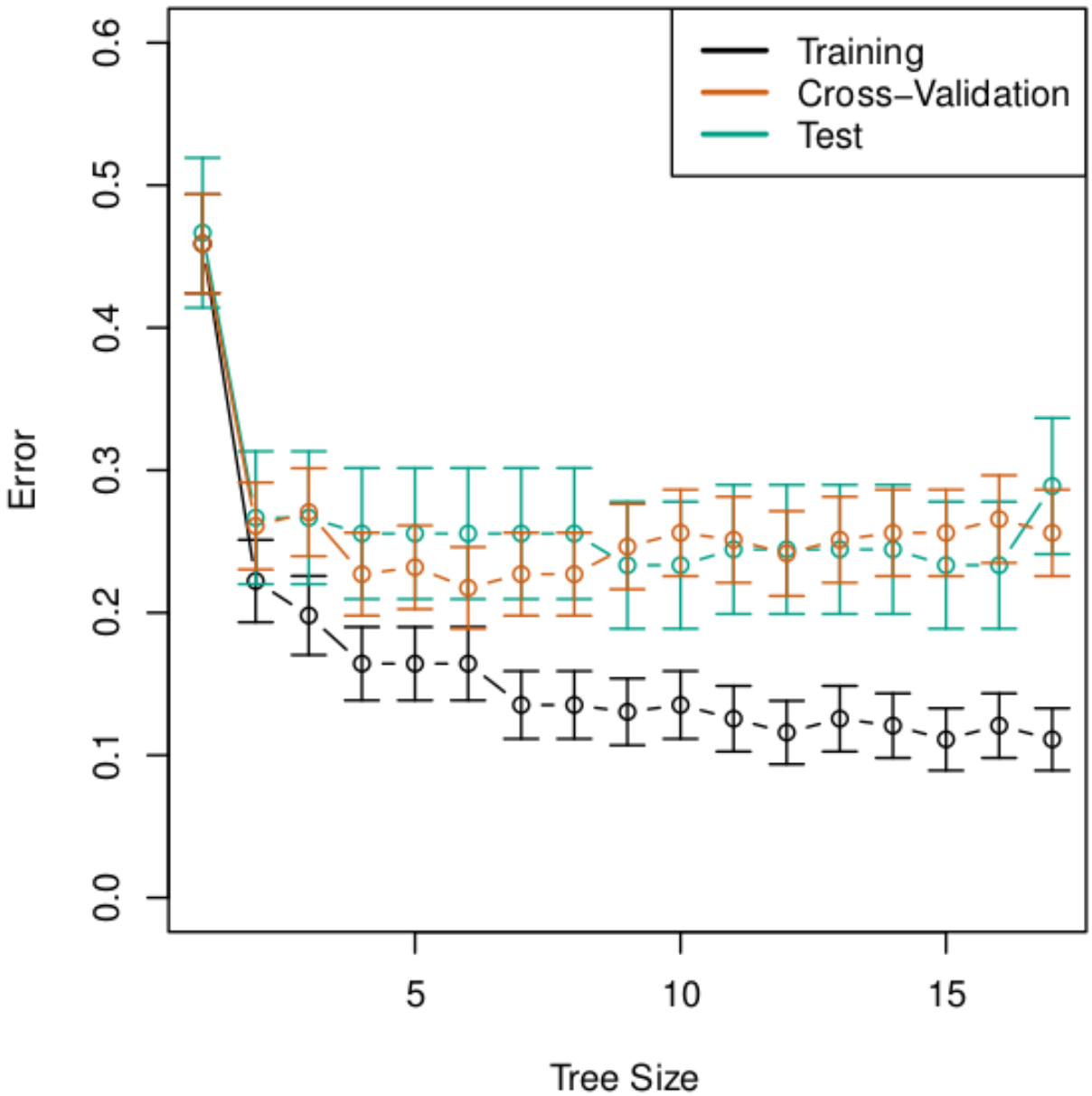
$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk} \tag{8.7}$$

Example: `Heart` classification dataset

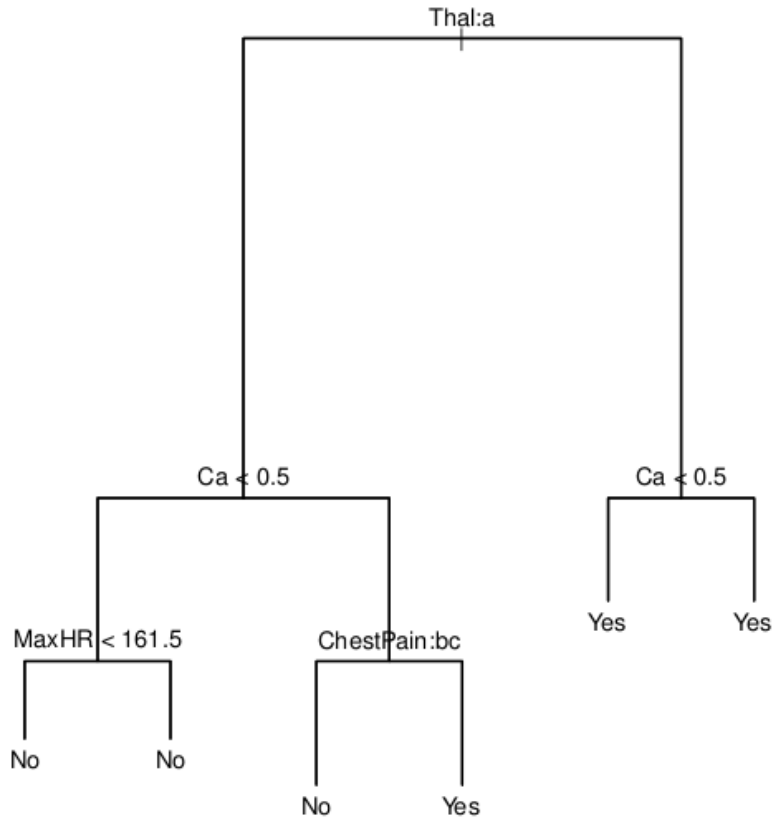


The full T_0 tree

Example: Heart classification dataset

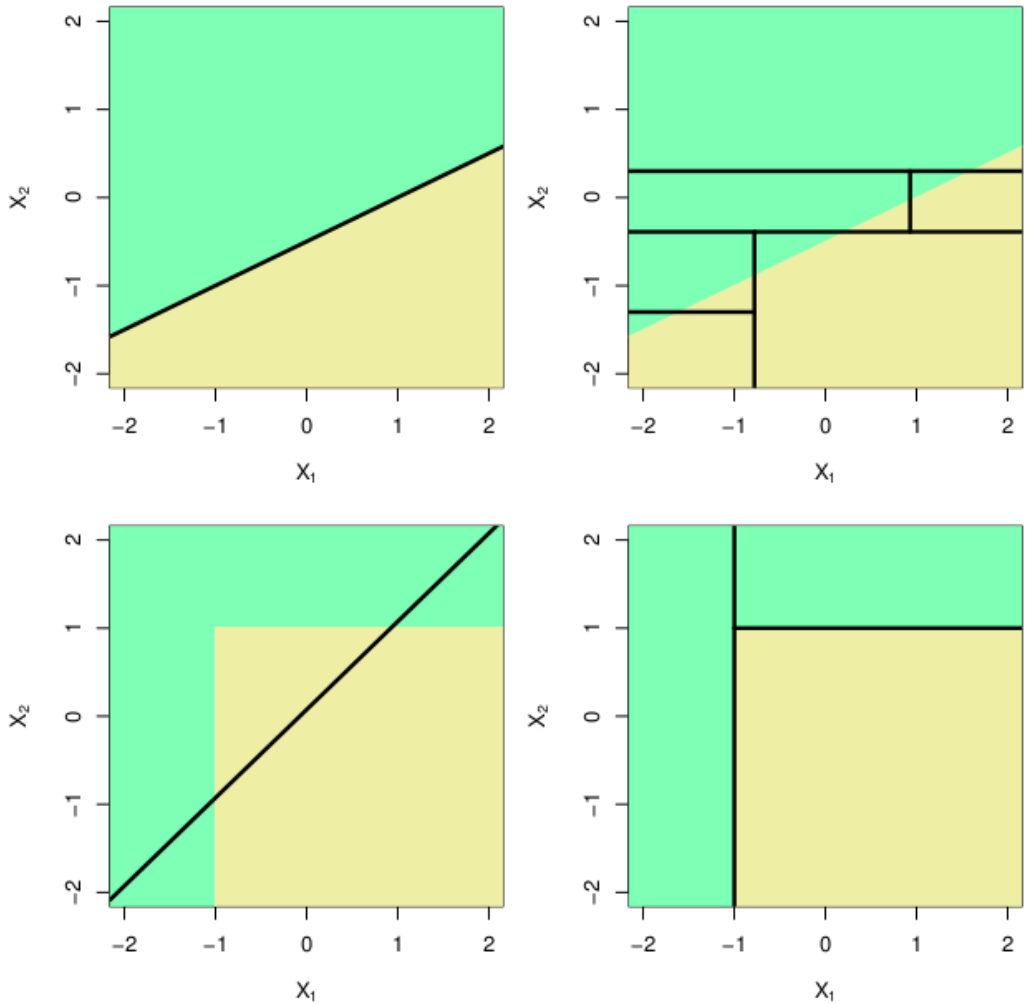


Example: `Heart` classification dataset



The chosen T through CV

Linear regression vs. decision trees



- True processes boundaries given in colors
- Left column: linear model; Right column: decision trees

Limitations of decision trees

- We saw that trees have very good interpretability
- They can be displayed graphically and are good to deal with qualitative attributes
- However, they often don't have a good accuracy
- Workarounds to improve predictive performance?

Ensemble methods

- An ensemble model use **multiple learning models** instead of a single one

- It often yields better predictive performance than using any of its composing models alone

- Some types of ensemble methods:

  1. Bagging
  2. Random forest
  3. Boosting

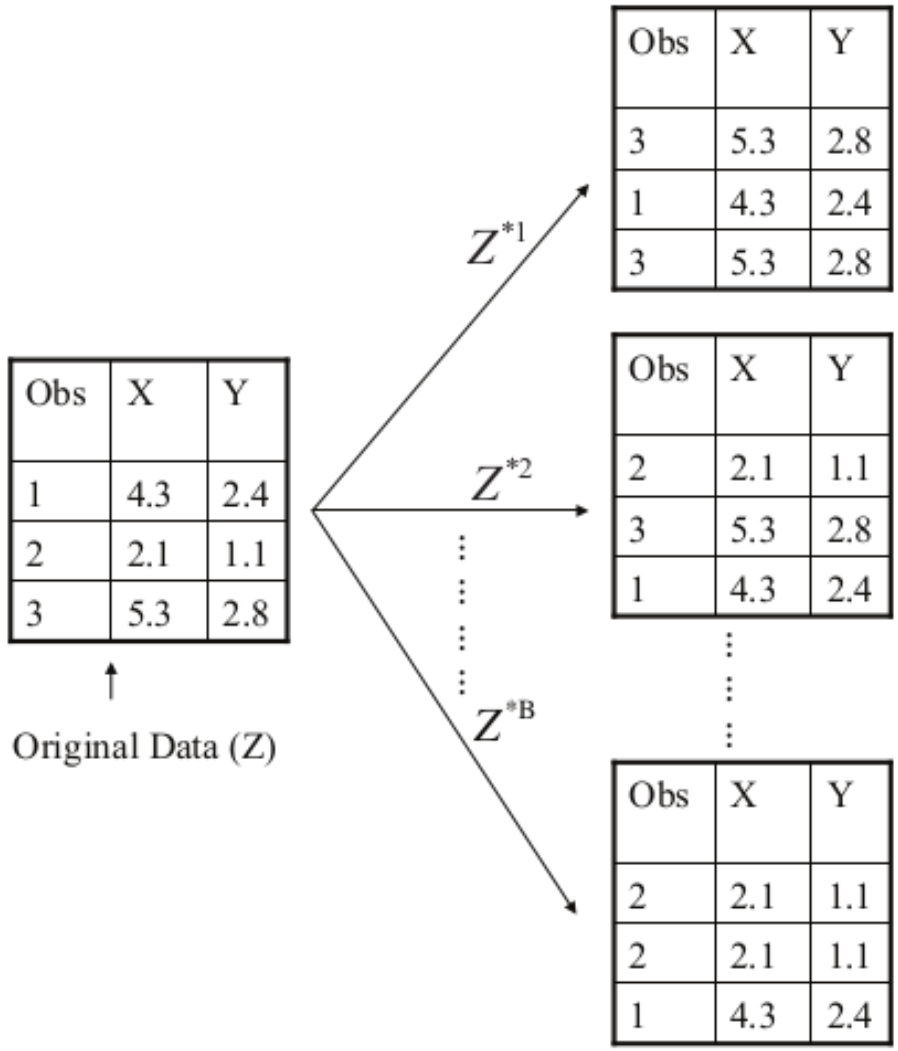
1. Baggings

- Also known as "bootstrap aggregations", they are a method for reducing the variance of a learning model
- They are a general-purpose method, however especially useful in the context of decision trees
- They are based in the fact that averaging results from different datasets reduces model variance

Obtaining more datasets

- How to obtain more datasets?
- We can use an approach called **bootstrap**
- In bootstrap, we take repeated samples *with replacement* from our training dataset to generate new datasets

Bootstrap example for $N = 3$ datapoints (observations)



Designing a bagging model

1. We produce B bootstrapped datasets

2. The bth bootstrapped training set is used train the classifier/regression $\hat{f}^{*b}(x)$

3. Final result is obtained by taking the most often classification among $\hat{f}^{*1}(x), \ldots, \hat{f}^{*B}(x)$ or, for regression, by averaging all the predictions:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{*b}(x)$$

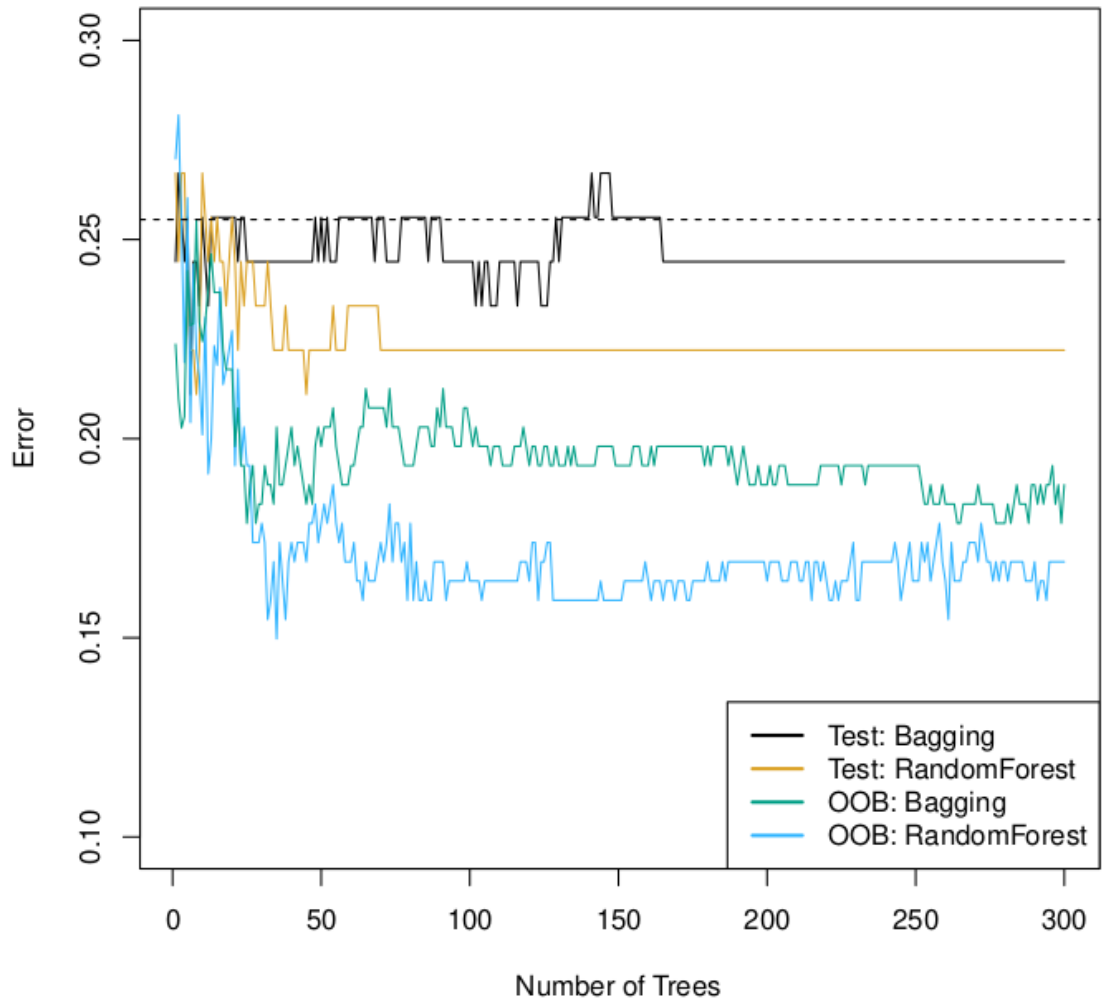
Out-of-bag error estimation

- If we generate B bootstrapped datasets Z^{*b} using a dataset Z of size n, one can show that, on average, each Z^{*b} makes use of $\frac{2}{3}n$ observations
- That means that, on average, each bagged tree doesn't use $\frac{1}{3}n$ observations for training
- For each x in Z, we can classify it using bagged trees that don't use x for training and average the results
- For a large value of B, this procedure is akin to leave-one-out CV

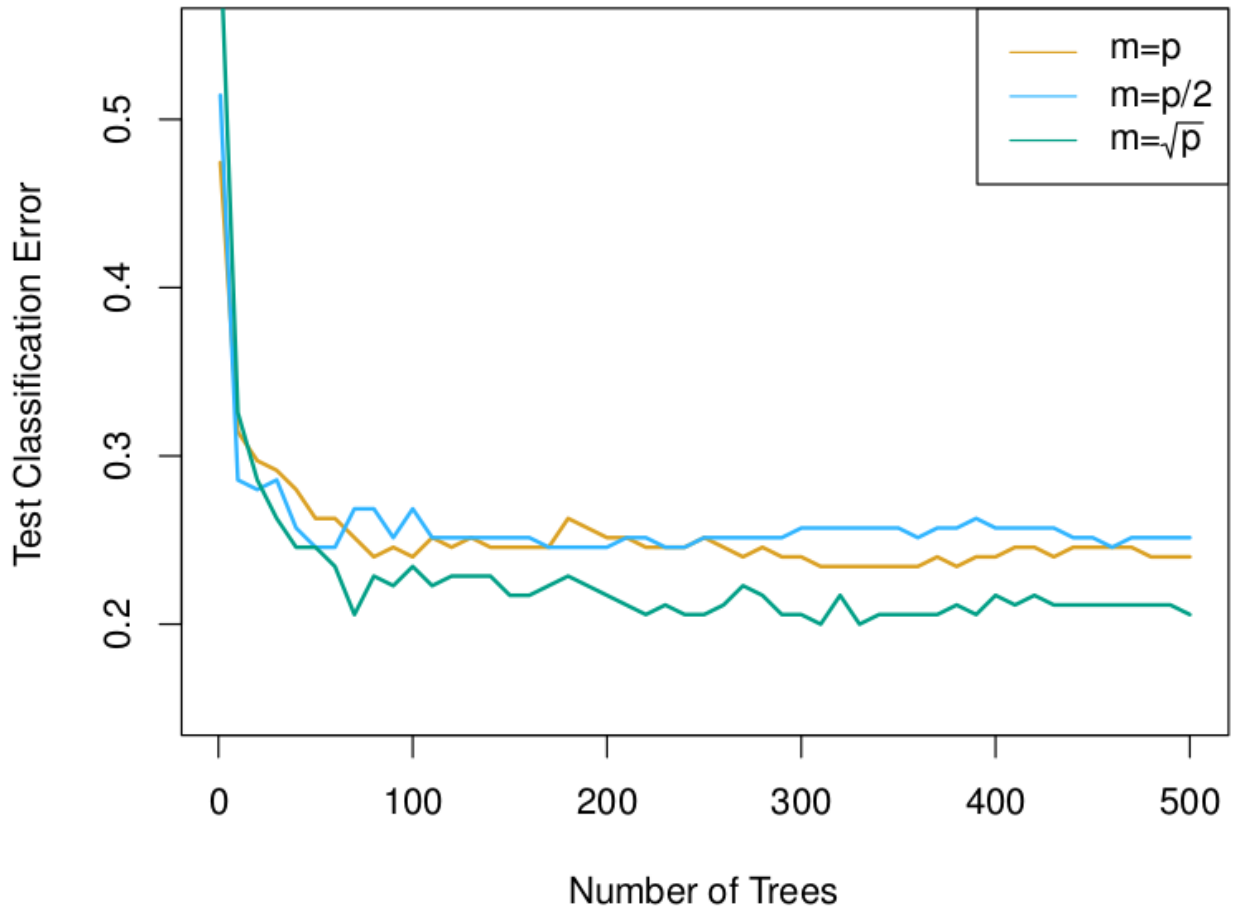
2. Random forests

- Improvement over bagged trees, based on a change in the tree-building process that decorrelates the trees
- This change reduces the variance when we average the trees
- The change is: at each iteration of the tree building process, a random subset of $x_1, \ldots, x_p$ is chosen, and the best split is defined only using variables of that subset
- Typical subset size is $\sqrt{p}$

Example 1: Heart dataset



Example 2: Gene expression dataset



3. Boosting

- Also a general general-purpose method that is very useful in the context of trees
- Instead of sampling the training dataset, Boosting uses a different strategy to build an ensemble
- In Boosting, the models are trained sequentially, using the information from previous models

3. Boosting

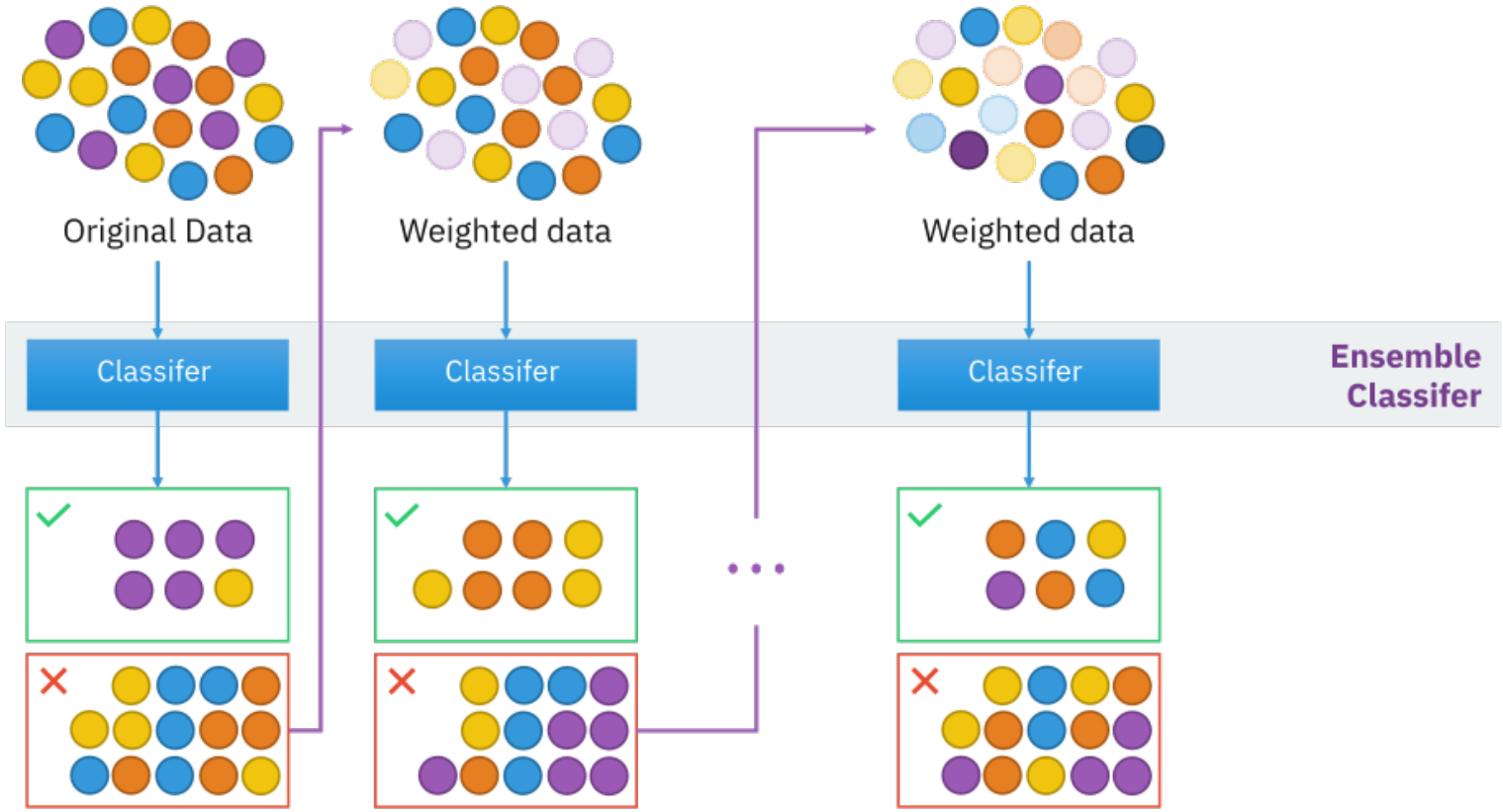


Figure: Boosting Ensemble scheme by Sirakorn, CC BY-SA 4.0, wikimedia id = 85888769

3. Boosting

- Boosting approach learns slowly. The trees are usually small and are determined by the parameter d.
- Given the current model, we fit a decision tree to the residuals from the model. We then add this new decision tree into the fitted function to update the residuals
- The training process forces the models to improve in areas where the others have failed, focusing on difficult parts

3. Boosting

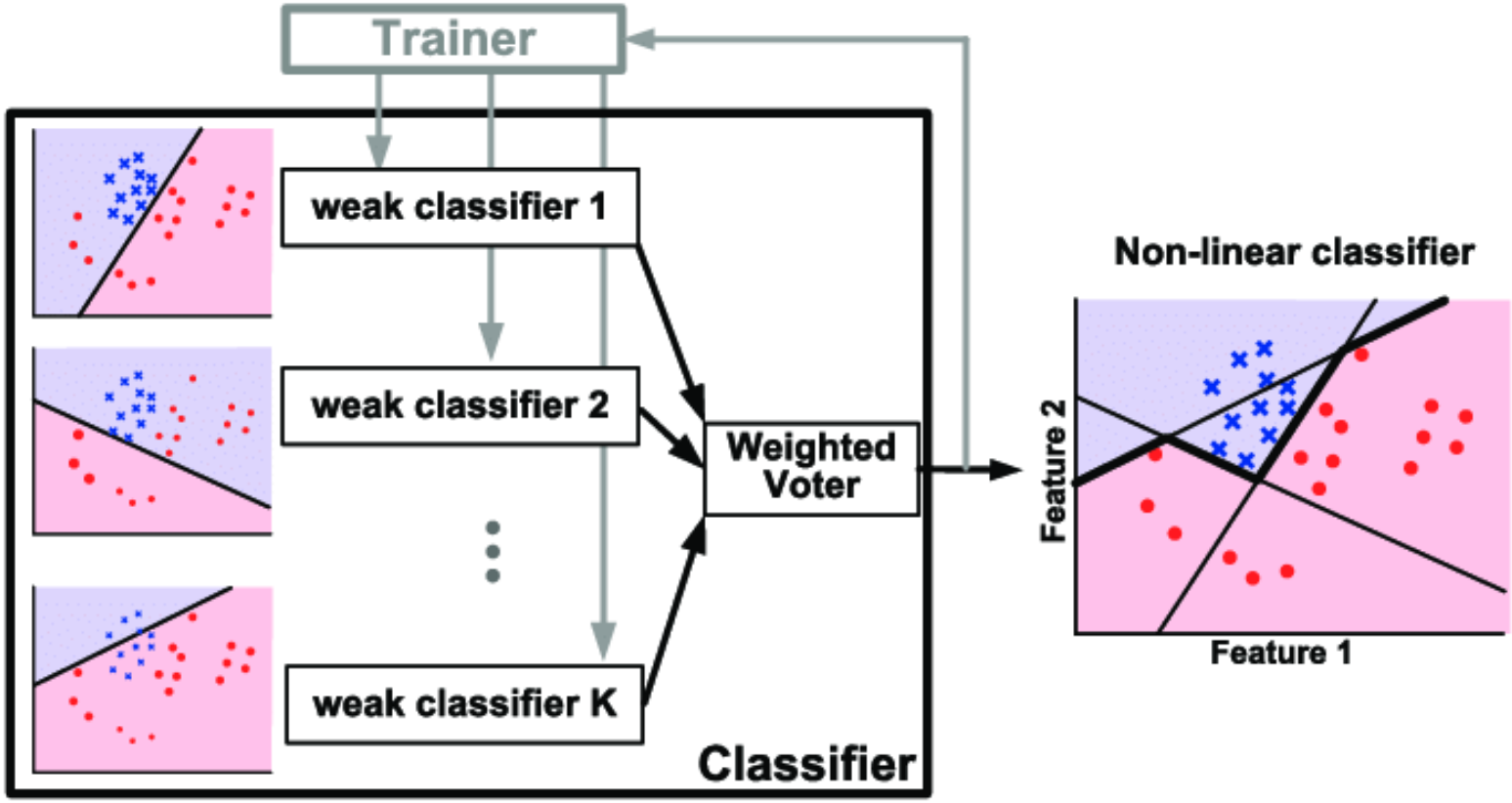


Figure: Result of fitting a boosting ensemble with weak classifiers in 2D feature space.

Realizing Low-Energy Classification Systems by Implementing Matrix Multiplication Directly Within an ADC - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Illustration-of-AdaBoost-algorithm-for-creating-a-strong-classifier-based-on-multiple_fig9_288699540 [accessed 29 Apr, 2024]

Algorithm 8.2: Boosting for Regression Trees

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, ..., B$, repeat:
   a) Fit a tree $\hat{f}^b$ with d splits ($d + 1$ terminal nodes) to the training data (X,r).
   b) Update $\hat{f}$ by adding in a shrunken version of the new tree:
   $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \tag{8.10}$$
   c) Update the residuals,
   $$r_i \leftarrow r_i - \lambda \hat{f}^b(x). \tag{8.11}$$
3. Output the boosted model,
$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x). \tag{8.12}$$

Variable importance

- In those ensemble methods, we can measure the importance of each variable for classification/regression
- For regression, we store the total amount of reduction of the objective function during splits on a given variable
- For classification, we store the total amount that Gini index is decreased during splits over a given variable
- At either case, we average results over all B trees; a large value indicates an important variable

Variable importance example: `Heart` dataset